

Encoding Categorical Data

Module 26 Study Notes • Feature Engineering

1. Introduction to Categorical Encoding

In previous modules, we focused on handling and scaling numerical data. However, in real-world machine learning projects, a significant portion of your dataset will exist in text format (categorical data). Because machine learning algorithms (such as K-Nearest Neighbors, Linear Regression, etc.) rely purely on mathematical computations, they can only interpret numbers. Therefore, transforming categorical string data into numerical formats is an indispensable step in the Feature Engineering pipeline.

2. Understanding Categorical Data Types

Before writing any code or applying an encoding technique, you must critically evaluate and classify your categorical variables into one of two groups. Applying the wrong encoding method can severely degrade your algorithm's predictive performance.

A. Nominal Data

Nominal data consists of categories that possess **no intrinsic mathematical order or hierarchy**. Every category is theoretically and mathematically equal to the others.

- **Examples:** States (West Bengal, Maharashtra, Karnataka), Engineering Branches (Computer Science, IT, Mechanical), Gender (Male, Female), or Car Brands.
- **Encoding Strategy:** You cannot assign sequential numbers like 1, 2, and 3 to these variables. If you do, the model will falsely infer that category 3 is "greater than" or "more important than" category 1. Instead, we use **One-Hot Encoding** (which will be covered extensively in the next module).

B. Ordinal Data

Ordinal data consists of categories where an **inherent order or ranking definitively exists**. While the exact mathematical difference between the values might not be perfectly quantifiable, the hierarchical progression is clear.

- **Examples:** Education Level (High School < Undergrad < Postgrad), Customer Reviews (Poor < Average < Good), or Income Brackets (Low, Medium, High).

- **Encoding Strategy:** We use **Ordinal Encoding** to map these text values to ranked integers (e.g., 0, 1, 2). This ensures that the algorithm recognizes and preserves the relationship and scale of the data.

3. Ordinal Encoding (For Input Features - X)

When dealing with input features (the independent variables that your model learns from, traditionally denoted as **X**), Scikit-Learn provides the powerful **OrdinalEncoder** class.

The Danger of Random Assignment: By default, if you do not explicitly specify the order of your categories, **OrdinalEncoder** will assign integers based on alphabetical sorting. For example, "Average", "Good", "Poor" might become 0, 1, 2. This completely destroys the logical ordinal relationship. You must explicitly pass the **categories** parameter as a list of lists.

Implementation Workflow

Consider a dataset with two ordinal input columns: 'Review' (Poor, Average, Good) and 'Education' (School, UG, PG).

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OrdinalEncoder

# 1. Train-Test Split (ALWAYS split before encoding to prevent Data Leakage)
X_train, X_test, y_train, y_test = train_test_split(df.iloc[:, 0:2], df.iloc[:, -1], test_size=0.2)

# 2. Define the explicit ascending order for each column
review_order = ['Poor', 'Average', 'Good']
edu_order = ['School', 'UG', 'PG']

# 3. Initialize the Encoder with the specific hierarchy
oe = OrdinalEncoder(categories=[review_order, edu_order])

# 4. Fit on Train data, Transform on both Train and Test data
X_train_encoded = oe.fit_transform(X_train)
X_test_encoded = oe.transform(X_test)
```

4. Label Encoding (For Target Variable - y)

While `OrdinalEncoder` is strictly meant for the input features, Scikit-Learn provides a separate, distinct class called `LabelEncoder` designed exclusively for the **target or output variable** (usually denoted as `y`).

If you are working on a Classification problem where the target variable is categorical text (e.g., "Purchased": Yes/No, or "Spam": True/False), you must encode it before passing it into the model.

Why use a separate class?

- **Array Dimensionality:** `OrdinalEncoder` is built to handle 2D arrays (dataframes containing multiple columns simultaneously). Conversely, `LabelEncoder` is strictly built to handle 1D arrays (a single target column or vector).
- **Documentation Rule:** Scikit-Learn's official documentation explicitly warns against using `LabelEncoder` on input features (X). Misusing it can lead to pipeline incompatibilities.

Implementation Workflow

```
from sklearn.preprocessing import LabelEncoder

# 1. Initialize the Encoder (No need to pass categories for the target)
le = LabelEncoder()

# 2. Fit on training labels, transform both train and test labels
y_train_encoded = le.fit_transform(y_train)
y_test_encoded = le.transform(y_test)
```

5. Summary of Best Practices

To avoid critical pipeline errors during the Feature Engineering phase, follow this mental checklist:

1. **Split First:** Always partition your data into training and testing sets before initializing or fitting any encoders. Fitting an encoder on the entire dataset exposes testing data features to the training process, causing Data Leakage.
2. **Isolate Columns:** At this manual stage of learning, since different columns require entirely different encodings (e.g., Ordinal vs. One-Hot), you must manually extract the ordinal

columns, apply the `OrdinalEncoder`, and then carefully concatenate the numpy arrays back together. (This tedious process will be elegantly automated in the future using the `ColumnTransformer`).

3. **Check for Garbage Data:** Ensure that your ordinal categorical columns do not accidentally contain hidden numerical junk, missing strings, or extra whitespaces before passing them into the `categories` list, as this will throw a `ValueError`.